TUTTI ESCOLA DE MÚSICA &j

ARQUITETURA DE BANCO DE DADOS

ESTRUTURA COMPLETA POSTGRESQL & SUPABASE

Documentação Técnica Oficial, Modelagem Relacional, Triggers, Views, RLS e Seeds

RESUMO EXECUTIVO DO SISTEMA

- **Banco de Dados:** PostgreSQL 15+ hospedado no Supabase
- **Projeto Supabase:** https://viwxalwpvuvgcyxsdjbi.supabase.co
- **Versão da Arquitetura:** 2.0 (Relacional, Multi-Ambiente com RLS)
- **Ambientes Atendidos:** Desenvolvedor / Administrador, Professor e Aluno
- **Total de Tabelas:** 18 tabelas relacionais com chaves estrangeiras
- **Total de Views:** 2 views analíticas (Destaques e Ranking de Alunos)
- **Políticas RLS:** 25 políticas granulares de segurança por usuário
- **Triggers e Funções:** Criação de perfis, auditoria e atualização de timestamps
- **Data de Geração:** 01 de novembro de 2026

Tutti Escola de Música - Todos os direitos reservados.

Índice da Modelagem de Dados

O banco de dados foi projetado em conformidade com as regras ACID e organizado nos seguintes módulos temáticos:

[00] **Extensões PostgreSQL** — uuid-ossp e pgcrypto para criptografia e identificadores globais.

[01] **Usuários, Perfis e Autenticação** — public.usuarios e public.profiles com triggers automáticos ao cadastrar.

[02] **Planos de Assinatura & Alunos** — public.planos (Free, Pro, Ultra) e assinaturas ativas com histórico financeiro.

[03] **Catálogo de Instrumentos** — public.instrumentos com afinações (Eb, Bb, C), famílias e professores.

[04] **Cursos, Aulas & Progresso** — public.cursos, public.aulas e acompanhamento individual lesson_progress.

[05] **Biblioteca de Partituras & Favoritos** — public.partituras com arranjos, cifras, playalongs e favoritos_partituras.

[06] **Estúdio Virtual & Gravações** — public.practice_logs para salvar áudios, afinação e BPM de estudo.

[07] **Mural da Comunidade Musical** — public.community_posts, likes e relacionamentos entre alunos.

[08] **Loja Tutti & Pedidos** — public.produtos_loja, cupons de desconto e pedidos de instrumentos/acessórios.

[09] **Masterclasses ao Vivo & RSVPs** — public.masterclasses e controle de presença em workshops.

[10] **Views Consolidadas & Ranking** — v_servicos_destaque e v_ranking_alunos ordenado por XP e constância.

[11] **Segurança & Row Level Security (RLS)** — 25 políticas restritivas por usuário autenticado e permissões públicas.

[12] **Seeds & Dados Iniciais Oficiais** — Carga de planos, instrumentos, partituras clássicas e materiais didáticos.

[13] **Gestão Multi-Ambiente (Dev, Prof, Aluno)** — solicitacoes_professores, marcacoes_aulas (com Google Meet) e auditoria.

Código SQL Completo (PostgreSQL DDL & DML)

Este código pode ser executado diretamente no SQL Editor do Supabase para criar todo o banco.

```sql
-- ============================================================================
-- TUTTI ESCOLA DE MÚSICA &j - ARQUITETURA COMPLETA DE BANCO DE DADOS (SUPABASE)
-- Projeto: https://viwxalwpvuvgcyxsdjbi.supabase.co
-- Versão: 2.0 (Completa, Relacional, RLS e Seeds)
-- ============================================================================
-- Este script pode ser executado no SQL Editor do painel do Supabase.
-- Ele cria toda a estrutura de tabelas, relacionamentos, triggers automáticos,
-- views consolidadas, políticas de segurança (RLS) e dados iniciais (seeds).
-- ============================================================================

-- 0. EXTENSÕES DO POSTGRESQL
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

-- ============================================================================
-- 1. TABELA PRINCIPAL DE USUÁRIOS E PERFIS (public.usuarios)
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.usuarios (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE UNIQUE,
  nome_completo TEXT NOT NULL,
  bio TEXT DEFAULT 'Estudando Saxofone e Harmonia no Tutti Escola de Música Ø<ß¶',
  avatar_url TEXT,
  current_instrument TEXT DEFAULT 'Saxofone Alto',
  level TEXT DEFAULT 'Intermediário' CHECK (level IN ('Iniciante', 'Intermediário', 'Avançado')),
  streak_days INTEGER DEFAULT 14,
  xp INTEGER DEFAULT 450,
  practice_hours INTEGER DEFAULT 26,
  completed_lessons INTEGER DEFAULT 8,
  role TEXT DEFAULT 'student' CHECK (role IN ('student', 'teacher', 'admin'))
);

CREATE INDEX IF NOT EXISTS idx_usuarios_user_id ON public.usuarios(user_id);

-- Trigger para atualizar updated_at automaticamente
CREATE OR REPLACE FUNCTION public.set_current_timestamp_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_usuarios_updated_at ON public.usuarios;
CREATE TRIGGER trg_usuarios_updated_at
  BEFORE UPDATE ON public.usuarios
  FOR EACH ROW EXECUTE FUNCTION public.set_current_timestamp_updated_at();

-- Trigger automático para criar o perfil em public.usuarios sempre que um aluno se cadastrar no auth.users
CREATE OR REPLACE FUNCTION public.handle_new_user()
RETURNS TRIGGER AS $$
BEGIN
  INSERT INTO public.usuarios (user_id, nome_completo, avatar_url, bio)
  VALUES (
    NEW.id,
    COALESCE(
      NEW.raw_user_meta_data->>'full_name',
      NEW.raw_user_meta_data->>'display_name',
      NEW.raw_user_meta_data->>'name',
      split_part(NEW.email, '@', 1)
    ),
    COALESCE(
      NEW.raw_user_meta_data->>'avatar_url',
      NEW.raw_user_meta_data->>'picture',
      NULL
    ),
    'Estudando no Tutti Escola de Música Ø<ß¶'
  )
  ON CONFLICT (user_id) DO UPDATE SET
    nome_completo = EXCLUDED.nome_completo,
    avatar_url = COALESCE(EXCLUDED.avatar_url, public.usuarios.avatar_url);
  RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

DROP TRIGGER IF EXISTS on_auth_user_created ON auth.users;
CREATE TRIGGER on_auth_user_created
  AFTER INSERT ON auth.users
  FOR EACH ROW EXECUTE FUNCTION public.handle_new_user();

-- Tabela de compatibilidade (profiles)
CREATE TABLE IF NOT EXISTS public.profiles (
  id UUID REFERENCES auth.users(id) ON DELETE CASCADE PRIMARY KEY,
  email TEXT,
  display_name TEXT,
  photo_url TEXT,
```

```
  role TEXT DEFAULT 'student',
  current_instrument TEXT DEFAULT 'Saxofone',
  level TEXT DEFAULT 'Intermediário',
  streak_days INTEGER DEFAULT 14,
  xp INTEGER DEFAULT 450,
  practice_hours INTEGER DEFAULT 26,
  completed_lessons INTEGER DEFAULT 8,
  bio TEXT DEFAULT 'Estudando no Tutti &j',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================================
-- 2. PLANOS DE ASSINATURA & ASSINATURAS DE ALUNOS
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.planos (
  id TEXT PRIMARY KEY, -- 'free', 'pro', 'ultra', 'ultra_trimestral', 'ultra_semestral'
  nome TEXT NOT NULL,
  preco NUMERIC(10, 2) NOT NULL,
  recorrencia TEXT NOT NULL CHECK (recorrencia IN ('gratis', 'vitalicio', 'mensal', 'trimestral',
'semestral')),
  descricao TEXT,
  beneficios JSONB DEFAULT '[]'::jsonb,
  badge TEXT,
  ativo BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.assinaturas_usuarios (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  plano_id TEXT REFERENCES public.planos(id),
  status TEXT DEFAULT 'ativo' CHECK (status IN ('ativo', 'cancelado', 'pendente')),
  data_inicio TIMESTAMPTZ DEFAULT NOW(),
  data_fim TIMESTAMPTZ,
  valor_pago NUMERIC(10, 2) DEFAULT 0.00,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- ============================================================================
-- 3. CATÁLOGO DE INSTRUMENTOS
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.instrumentos (
  id TEXT PRIMARY KEY, -- 'sax', 'flauta', 'trombone', 'trompete', 'teclado', 'violao'
  nome TEXT NOT NULL,
  afinacao TEXT NOT NULL, -- 'Eb', 'Bb', 'C', etc.
  familia TEXT NOT NULL CHECK (familia IN ('Sopro / Madeiras', 'Sopro / Metais', 'Teclas', 'Cordas', 'Voz',
'Percussão')),
  emoji TEXT,
  imagem_url TEXT,
  professor_responsavel TEXT,
  descricao TEXT,
  tecnicas_foco JSONB DEFAULT '[]'::jsonb,
  ordem INTEGER DEFAULT 0,
  ativo BOOLEAN DEFAULT true
);

-- ============================================================================
-- 4. CURSOS, AULAS & PROGRESSO DO ALUNO
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.cursos (
  id TEXT PRIMARY KEY,
  instrumento_id TEXT REFERENCES public.instrumentos(id) ON DELETE CASCADE,
  titulo TEXT NOT NULL,
  descricao TEXT,
  nivel TEXT DEFAULT 'Iniciante',
  capa_url TEXT,
  professor TEXT,
  total_aulas INTEGER DEFAULT 12,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.aulas (
  id SERIAL PRIMARY KEY,
  instrumento_id TEXT REFERENCES public.instrumentos(id) ON DELETE CASCADE,
  curso_id TEXT REFERENCES public.cursos(id) ON DELETE SET NULL,
  titulo TEXT NOT NULL,
  modulo TEXT NOT NULL,
  duracao TEXT,
  nivel TEXT DEFAULT 'Iniciante',
  professor TEXT,
  video_url TEXT,
  resumo TEXT,
  pontos_chave JSONB DEFAULT '[]'::jsonb,
  cifras JSONB DEFAULT '[]'::jsonb,
  tablatura TEXT,
  exercicios_praticos JSONB DEFAULT '[]'::jsonb,
  ordem INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.lesson_progress (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```sql
  user_id TEXT NOT NULL,
  lesson_id TEXT NOT NULL,
  instrument_id TEXT,
  completed BOOLEAN DEFAULT false,
  completed_tasks JSONB DEFAULT '[]'::jsonb,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, lesson_id)
);

CREATE INDEX IF NOT EXISTS idx_lesson_progress_user ON public.lesson_progress(user_id);

-- ============================================================================
-- 5. BIBLIOTECA DE PARTITURAS & FAVORITOS
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.partituras (
  id SERIAL PRIMARY KEY,
  titulo TEXT NOT NULL,
  compositor TEXT NOT NULL,
  instrumento TEXT NOT NULL,
  tonalidade TEXT NOT NULL,
  tom_original TEXT,
  nivel TEXT DEFAULT 'Intermediário' CHECK (nivel IN ('Iniciante', 'Intermediário', 'Avançado')),
  genero TEXT DEFAULT 'Bossa Nova',
  tempo_bpm INTEGER DEFAULT 120,
  compasso TEXT DEFAULT '4/4',
  descricao TEXT,
  arquivo_pdf_url TEXT,
  storage_path TEXT,
  is_featured BOOLEAN DEFAULT false,
  featured_order INTEGER DEFAULT 0,
  file_size_bytes BIGINT DEFAULT 0,
  created_by TEXT,
  audio_playalong_url TEXT,
  youtube_url TEXT,
  cifras JSONB DEFAULT '[]'::jsonb,
  melodia_notas JSONB DEFAULT '[]'::jsonb,
  visualizacoes INTEGER DEFAULT 0,
  downloads INTEGER DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.favoritos_partituras (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  partitura_id INTEGER REFERENCES public.partituras(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, partitura_id)
);

-- ============================================================================
-- 6. GRAVAÇÕES & ESTÚDIO DE AUTOAVALIAÇÃO
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.practice_logs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id TEXT NOT NULL,
  instrument TEXT NOT NULL,
  duration_seconds INTEGER DEFAULT 0,
  audio_data_url TEXT,
  notes TEXT,
  bpm_praticado INTEGER,
  afinacao_precisao NUMERIC(5, 2),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_practice_logs_user ON public.practice_logs(user_id);

-- ============================================================================
-- 7. MURAL DA COMUNIDADE MUSICAL & COMENTÁRIOS
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.community_posts (
  id TEXT PRIMARY KEY DEFAULT ('post_' || extract(epoch from now())::bigint::text),
  author_id TEXT NOT NULL,
  author_name TEXT NOT NULL,
  author_avatar TEXT,
  instrument TEXT DEFAULT 'Saxofone',
  content TEXT NOT NULL,
  image_url TEXT,
  video_url TEXT,
  likes INTEGER DEFAULT 0,
  comments JSONB DEFAULT '[]'::jsonb,
  shares INTEGER DEFAULT 0,
  tag TEXT DEFAULT '#Tutti',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_community_posts_created ON public.community_posts(created_at DESC);

CREATE TABLE IF NOT EXISTS public.post_likes (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  post_id TEXT REFERENCES public.community_posts(id) ON DELETE CASCADE,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(post_id, user_id)
```

```
);

CREATE TABLE IF NOT EXISTS public.amizades (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  amigo_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  status TEXT DEFAULT 'aceito' CHECK (status IN ('pendente', 'aceito', 'bloqueado')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, amigo_id)
);

-- ============================================================================
-- 8. LOJA TUTTI, PRODUTOS & PEDIDOS
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.produtos_loja (
  id SERIAL PRIMARY KEY,
  nome TEXT NOT NULL,
  preco NUMERIC(10, 2) NOT NULL,
  preco_antigo NUMERIC(10, 2),
  categoria TEXT NOT NULL CHECK (categoria IN ('Acessórios', 'Instrumentos', 'Vestuário', 'Livros',
'Partituras')),
  imagem_url TEXT,
  avaliacao NUMERIC(3, 2) DEFAULT 4.9,
  total_avaliacoes INTEGER DEFAULT 25,
  badge TEXT,
  descricao TEXT,
  em_estoque BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.cupons_desconto (
  codigo TEXT PRIMARY KEY,
  percentual_desconto NUMERIC(5, 2) NOT NULL,
  valido_ate TIMESTAMPTZ,
  ativo BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.orders (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL,
  user_email TEXT NOT NULL,
  items JSONB NOT NULL,
  subtotal NUMERIC(10, 2) NOT NULL,
  discount NUMERIC(10, 2) DEFAULT 0,
  total NUMERIC(10, 2) NOT NULL,
  coupon TEXT,
  payment_method TEXT NOT NULL,
  status TEXT DEFAULT 'confirmed' CHECK (status IN ('pending', 'confirmed', 'shipped', 'cancelled')),
  shipping_address TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_orders_user ON public.orders(user_id);

-- ============================================================================
-- 9. MASTERCLASSES AO VIVO & INSCRIÇÕES
-- ============================================================================
CREATE TABLE IF NOT EXISTS public.masterclasses (
  id SERIAL PRIMARY KEY,
  titulo TEXT NOT NULL,
  instrutor TEXT NOT NULL,
  instrumento TEXT NOT NULL,
  data_evento DATE NOT NULL,
  horario TEXT NOT NULL,
  vagas_maximas INTEGER DEFAULT 200,
  inscritos_count INTEGER DEFAULT 0,
  nivel TEXT DEFAULT 'Todos os Níveis',
  status TEXT DEFAULT 'Inscrições Abertas' CHECK (status IN ('Inscrições Abertas', 'Hoje', 'Encerrado')),
  descricao TEXT,
  link_ao_vivo TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS public.masterclass_rsvps (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id TEXT NOT NULL,
  masterclass_id INTEGER REFERENCES public.masterclasses(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  registered_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, masterclass_id)
);

-- ============================================================================
-- 10. VIEWS CONSOLIDADAS & DASHBOARD
-- ============================================================================

-- View de serviços e cursos em destaque solicitada
CREATE OR REPLACE VIEW public.v_servicos_destaque AS
SELECT
  'curso_sax' AS id,
  'Saxofone Express' AS titulo,
  'Do zero à improvisação no Saxofone Alto e Tenor' AS descricao,
```

```sql
    'Instrumentos de Sopro' AS categoria,
    4.9 AS avaliacao,
    true AS em_destaque
  UNION ALL
  SELECT
    'curso_flauta' AS id,
    'Flauta Transversal & Respiração' AS titulo,
    'Técnica diafragmática, embocadura e afinação' AS descricao,
    'Sopro / Madeiras' AS categoria,
    5.0 AS avaliacao,
    true AS em_destaque
  UNION ALL
  SELECT
    'plano_pro' AS id,
    'Assinatura Tutti Pro Vitalício' AS titulo,
    'Mais de 1000 partituras interativas, estúdio e play-alongs sem mensalidades' AS descricao,
    'Planos de Ensino' AS categoria,
    4.98 AS avaliacao,
    true AS em_destaque;

-- View do Ranking de Alunos por XP
CREATE OR REPLACE VIEW public.v_ranking_alunos AS
SELECT
  id,
  nome_completo,
  avatar_url,
  current_instrument,
  level,
  xp,
  streak_days,
  completed_lessons,
  RANK() OVER (ORDER BY xp DESC) AS posicao
FROM public.usuarios
WHERE role = 'student';

-- ============================================================================
-- 11. ROW LEVEL SECURITY (RLS) - POLÍTICAS DE ACESSO
-- ============================================================================
ALTER TABLE public.usuarios ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.planos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.assinaturas_usuarios ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.instrumentos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cursos ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.aulas ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.lesson_progress ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.partituras ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.favoritos_partituras ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.practice_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.community_posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.post_likes ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.amizades ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.produtos_loja ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.cupons_desconto ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.masterclasses ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.masterclass_rsvps ENABLE ROW LEVEL SECURITY;

-- Catálogos públicos (Leitura livre)
CREATE POLICY "Leitura de planos" ON public.planos FOR SELECT USING (true);
CREATE POLICY "Leitura de instrumentos" ON public.instrumentos FOR SELECT USING (true);
CREATE POLICY "Leitura de cursos" ON public.cursos FOR SELECT USING (true);
CREATE POLICY "Leitura de aulas" ON public.aulas FOR SELECT USING (true);
CREATE POLICY "Leitura de partituras" ON public.partituras FOR SELECT USING (true);
CREATE POLICY "Leitura de produtos da loja" ON public.produtos_loja FOR SELECT USING (true);
CREATE POLICY "Leitura de cupons ativos" ON public.cupons_desconto FOR SELECT USING (ativo = true);
CREATE POLICY "Leitura de masterclasses" ON public.masterclasses FOR SELECT USING (true);
CREATE POLICY "Leitura de perfis de alunos" ON public.usuarios FOR SELECT USING (true);
CREATE POLICY "Leitura de perfis compatibilidade" ON public.profiles FOR SELECT USING (true);
CREATE POLICY "Leitura de posts da comunidade" ON public.community_posts FOR SELECT USING (true);

-- Permissões do Usuário Autenticado sobre seus próprios dados
CREATE POLICY "Usuário insere seu perfil"
  ON public.usuarios FOR INSERT WITH CHECK (auth.uid() = user_id OR auth.uid() IS NULL);

CREATE POLICY "Usuário atualiza apenas seu perfil"
  ON public.usuarios FOR UPDATE USING (auth.uid() = user_id) WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Usuário consulta suas assinaturas"
  ON public.assinaturas_usuarios FOR SELECT USING (auth.uid() = user_id);

CREATE POLICY "Usuário consulta seu progresso de aulas"
  ON public.lesson_progress FOR SELECT USING (auth.uid()::text = user_id OR user_id = 'guest' OR auth.uid()
IS NOT NULL);

CREATE POLICY "Usuário salva seu progresso de aulas"
  ON public.lesson_progress FOR ALL USING (auth.uid()::text = user_id OR user_id = 'guest' OR auth.uid() IS
NOT NULL);

CREATE POLICY "Usuário gerencia suas partituras favoritas"
  ON public.favoritos_partituras FOR ALL USING (auth.uid() = user_id);

CREATE POLICY "Usuário salva gravações no estúdio"
  ON public.practice_logs FOR ALL USING (auth.uid()::text = user_id OR user_id = 'guest' OR auth.uid() IS
NOT NULL);
```

```
457
458  CREATE POLICY "Criação de postagens na comunidade"
459    ON public.community_posts FOR INSERT WITH CHECK (true);
460
461  CREATE POLICY "Curtidas de postagens da comunidade"
462    ON public.community_posts FOR UPDATE USING (true);
463
464  CREATE POLICY "Usuário curte postagens"
465    ON public.post_likes FOR ALL USING (auth.uid() = user_id);
466
467  CREATE POLICY "Usuário consulta seus pedidos"
468    ON public.orders FOR SELECT USING (auth.uid()::text = user_id OR user_id = 'guest' OR auth.uid() IS NOT
NULL);
469
470  CREATE POLICY "Usuário cria pedidos"
471    ON public.orders FOR INSERT WITH CHECK (true);
472
473  CREATE POLICY "Usuário se inscreve em masterclasses"
474    ON public.masterclass_rsvps FOR ALL USING (auth.uid()::text = user_id OR user_id = 'guest' OR auth.uid()
IS NOT NULL);
475
476  -- ============================================================================
477  -- 12. DADOS INICIAIS (SEEDS)
478  -- ============================================================================
479
480  -- Planos Oficiais da Tutti Escola de Música
481  INSERT INTO public.planos (id, nome, preco, recorrencia, descricao, beneficios, badge)
482  VALUES
483    ('free', 'Plano Gratuito', 0.00, 'gratis', 'Ideal para conhecer a metodologia e primeiros passos',
484     '["Aulas básicas demonstrativas", "Acesso ao metrônomo e afinador", "Biblioteca inicial de partituras",
"Maestro IA limitado (5 perguntas/semana)"]'::jsonb, NULL),
485    ('pro', 'Plano Tutti Pro Vitalício', 49.90, 'vitalicio', 'Acesso completo vitalício sem mensalidades',
486     '["Mais de 1000 partituras completas e interativas", "Estúdio virtual com digitações completas", "Play-
alongs sincronizados com áudio real", "Aulas do iniciante ao avançado", "Acesso vitalício sem cobrança
mensal"]'::jsonb, 'Mais Popular'),
487    ('ultra', 'Plano Tutti Ultra', 99.90, 'vitalicio', 'Experiência máxima de performance e mentoria',
488     '["Mais de 1000 partituras com arranjos exclusivos", "Acesso prioritário a masterclasses ao vivo com
concertistas", "Feedback de áudio das suas gravações", "Mentoria com o Maestro", "Garantia estendida e suporte
VIP"]'::jsonb, 'Maior Indicação')
489  ON CONFLICT (id) DO UPDATE SET
490    preco = EXCLUDED.preco,
491    recorrencia = EXCLUDED.recorrencia,
492    beneficios = EXCLUDED.beneficios,
493    badge = EXCLUDED.badge;
494
495  -- Instrumentos de Sopro e Base
496  INSERT INTO public.instrumentos (id, nome, afinacao, familia, emoji, professor_responsavel, descricao, ordem)
497  VALUES
498    ('sax', 'Saxofone Alto', 'Eb', 'Sopro / Madeiras', 'Ø<ß·', 'Maestro Lucas', 'Sonoridade encorpada, ideal para
Bossa Nova, Choro, MPB e Jazz.', 1),
499    ('sax_tenor', 'Saxofone Tenor', 'Bb', 'Sopro / Madeiras', 'Ø<ß·', 'Maestro Lucas', 'Timbre quente e aveludado,
presença marcante nos solos clássicos.', 2),
500    ('sax_soprano', 'Saxofone Soprano', 'Bb', 'Sopro / Madeiras', 'Ø<ß·', 'Maestro Lucas', 'Afinação expressiva e
brilhante, clássico de canções instrumentais.', 3),
501    ('flauta', 'Flauta Transversal', 'C', 'Sopro / Madeiras', 'Ø>Þˆ', 'Profª Mariana Sales', 'Respiração
diafragmática, fluidez harmônica e agilidade melódica.', 4),
502    ('trombone', 'Trombone de Vara', 'Bb/F', 'Sopro / Metais', 'Ø<ßº', 'Prof. Sérgio Metal', 'Contrapontos,
glissandos perfeitos e peso orquestral.', 5),
503    ('trompete', 'Trompete', 'Bb', 'Sopro / Metais', 'Ø<ßº', 'Prof. Roberto Soares', 'Ataque límpido, extensão
aguda brilhante e liderança do naipe.', 6),
504    ('teclado', 'Piano / Teclado', 'C', 'Teclas', 'Ø<ß¹', 'Profª Clara Vasconcelos', 'Base harmônica, leitura de
partituras e condução de acordes.', 7),
505    ('violao', 'Violão & Harmonia', 'C', 'Cordas', 'Ø<ß¸', 'Prof. Lucas Arruda', 'Harmonia funcional brasileira,
levadas de samba e bossa.', 8)
506  ON CONFLICT (id) DO UPDATE SET
507    nome = EXCLUDED.nome,
508    afinacao = EXCLUDED.afinacao,
509    familia = EXCLUDED.familia,
510    descricao = EXCLUDED.descricao;
511
512  -- Partituras Clássicas do Repertório Brasileiro e Internacional
513  INSERT INTO public.partituras (titulo, compositor, instrumento, tonalidade, tom_original, nivel, genero,
tempo_bpm, compasso, descricao, visualizacoes)
514  VALUES
515    ('Carinhoso', 'Pixinguinha & João de Barro', 'Saxofone Alto (Eb)', 'G Maior', 'F Maior', 'Intermediário',
'Choro', 76, '2/4', 'Arranjo com contraponto lírico clássico e modulações tradicionais brasileiras.', 1420),
516    ('Garota de Ipanema', 'Tom Jobim & Vinicius de Moraes', 'Saxofone Tenor (Bb)', 'F Maior', 'F Maior',
'Intermediário', 'Bossa Nova', 126, '4/4', 'A clássica progressão ii-V-I da Bossa Nova com síncopes marcadas.',
2890),
517    ('Autumn Leaves', 'Joseph Kosma', 'Saxofone Alto (Eb)', 'G Menor', 'G Menor', 'Iniciante', 'Jazz
Standard', 115, '4/4', 'Música ideal para iniciar a improvisação sobre ciclos de quintas.', 3150),
518    ('Brasileirinho', 'Waldir Azevedo', 'Flauta Transversal (C)', 'A Maior', 'A Maior', 'Avançado', 'Choro',
132, '2/4', 'Exercício supremo de agilidade e técnica de respiração circular.', 1890),
519    ('Wave', 'Tom Jobim', 'Flauta Transversal (C)', 'D Maior', 'D Maior', 'Intermediário', 'Bossa Nova', 130,
'4/4', 'Melodia fluida que exige afinação estável nas notas agudas.', 1220),
520    ('Na Baixa do Sapateiro', 'Ary Barroso', 'Trombone de Vara', 'Bb Maior', 'Bb Maior', 'Intermediário',
'Samba-Choro', 110, '2/4', 'Naipe de metais com posições rápidas de vara e apoio diafragmático.', 840);
521
522  -- Produtos Populares da Loja Tutti
523  INSERT INTO public.produtos_loja (nome, preco, preco_antigo, categoria, imagem_url, avaliacao,
total_avaliacoes, badge, descricao)
524  VALUES
525    ('Palheta Vandoren Java Red Sax Alto 2.5 (Caixa com 5)', 189.90, 219.00, 'Acessórios', 'https://
images.unsplash.com/photo-1511192336575-5a79af67a629?w=400&auto=format&fit=crop&q=80', 4.9, 87, 'Mais Vendido',
'Palhetas flexíveis com sonoridade brilhante e resposta imediata.'),
```

```
526    ('Boquilha Yamaha 4C
para Sax Alto', 289.00,
320.00, 'Acessórios', 'https://
images.unsplash.com/photo-15259
94886773-080587e161c2?w=400&aut
o=format&fit=crop&q=80', 5.0,
142, 'Recomendado', 'A
boquilha padrão de
conservatórios para afinação
limpa e fácil emissão.'),
527    ('Método Amadeu Russo
para Saxofone & Flauta',
69.90, 85.00, 'Livros',
'https://images.unsplash.com/
photo-1544716278-ca5e3f4abd8c?w
=400&auto=format&fit=crop&q=80'
, 4.8, 64, NULL, 'Exercícios
clássicos de sonoridade,
intervalos e digitações
cromáticas.'),
528    ('Suporte Dobrável de
Chão para Sax Alto e Tenor',
139.90, 169.00, 'Acessórios',
'https://images.unsplash.com/ph
oto-1514525253161-7a46d19cd819?
w=400&auto=format&fit=crop&q=80
', 4.9, 43, NULL, 'Estrutura
metálica reforçada com
proteção em veludo para evitar
arranhões.');
529
530  -- Cupom Inicial de Boas-
Vindas
531  INSERT INTO
public.cupons_desconto
(codigo, percentual_desconto,
valido_ate, ativo)
532  VALUES ('TUTTI10', 10.00,
NOW() + INTERVAL '365 days',
true)
533  ON CONFLICT (codigo) DO
NOTHING;
534
535  -- Masterclasses em
Destaque
536  INSERT INTO
public.masterclasses (titulo,
instrutor, instrumento,
data_evento, horario,
vagas_maximas,
inscritos_count, nivel,
status, descricao)
537  VALUES
538    ('Oficina de Saúde
Vocal e Alta Performance',
'Dra. Mariana Sales', 'Voz &
Canto', CURRENT_DATE, '20:00',
250, 189, 'Todos os Níveis',
'Hoje', 'Aprenda técnicas de
fonoaudiologia para cantar com
afinação e proteger as pregas
vocais.'),
539    ('Masterclass: O Som
Aveludado no Saxofone',
'Maestro Lucas', 'Saxofone
Alto & Tenor', CURRENT_DATE +
INTERVAL '5 days', '19:30',
200, 94, 'Intermediário',
'Inscrições Abertas',
'Segredos de embocadura,
garganta relaxada e controle
de dinâmicas pianíssimo.');
540
541  -- Postagens Iniciais do
Mural da Comunidade
542  INSERT INTO
public.community_posts (id,
author_id, author_name,
author_avatar, instrument,
content, likes, shares, tag)
543  VALUES
544    ('p_init_1',
'admin_maestro', 'Maestro
Lucas Ø<ß¼', 'https://
images.unsplash.com/photo-15070
03211169-0a1dd7228f2d?w=150&aut
o=format&fit=crop&q=80',
'Saxofone Alto & Harmonia',
'Sejam bem-vindos à Tutti
Escola de Música! Pratique ao
menos 20 minutos hoje: 5 min
de notas longas, 5 min de
escalas com metrônomo e 10 min
no seu repertório favorito.',
48, 12, '#DicaDoMaestro
```

```
#Saxofone'),
    ('p_init_2',
'aluno_clara', 'Clara
Vasconcelos', 'https://
images.unsplash.com/
photo-1494790108377-
be9c29b29330?w=150&auto=format&
fit=crop&q=80', 'Flauta
Transversal', 'Estudando o
arranjo de "Carinhoso" na
flauta! O estúdio virtual com
o afinador ajudou demais a
estabilizar a 3ª oitava.', 35,
6, '#FlautaTransversal
#Choro');

-- ======================
============================
======================

-- 12. TABELAS DE GESTÃO
MULTI-AMBIENTE: DESENVOLVEDOR,
PROFESSOR & ALUNO

-- ======================
============================
======================

-- Tabela de
Administradores /
Desenvolvedores Autorizados
-- Apenas contas
explicitamente listadas aqui
podem exercer privilégios de
desenvolvedor.
CREATE TABLE IF NOT
EXISTS public.administradores_a
utorizados (
    email TEXT PRIMARY KEY,
    descricao TEXT DEFAULT
'Desenvolvedor & Administrador
Oficial Tutti',
    criado_em TIMESTAMPTZ
DEFAULT NOW()
);

-- Seed do Desenvolvedor
Principal Autorizado
INSERT INTO public.adminis
tradores_autorizados (email,
descricao)
VALUES
('eliandrolucas.el@gmail.com',
'Desenvolvedor Autorizado e
Arquiteto de Software')
ON CONFLICT (email) DO
NOTHING;

-- Função de Segurança no
Backend para Identificação do
Desenvolvedor
CREATE OR REPLACE
FUNCTION public.is_admin()
RETURNS BOOLEAN AS $$
BEGIN
  RETURN (

LOWER(COALESCE(auth.jwt() ->>
'email', '')) IN (
      SELECT LOWER(email)
FROM public.administradores_aut
orizados
    ) OR
LOWER(COALESCE(auth.jwt() ->>
'email', '')) =
'eliandrolucas.el@gmail.com'
  );
END;
$$ LANGUAGE plpgsql
SECURITY DEFINER;

-- Trigger de Proteção de
Perfis: impede auto-promoção
de role e approval_status
CREATE OR REPLACE
FUNCTION
public.protect_user_roles()
RETURNS TRIGGER AS $$
BEGIN
  -- Se quem está
atualizando NÃO é
administrador autorizado,
mantém o role e o
approval_status anteriores
  IF NOT
public.is_admin() THEN
    NEW.role := OLD.role;
```

```
583
NEW.approval_status :=
OLD.approval_status;
584    END IF;
585    RETURN NEW;
586  END;
587  $$ LANGUAGE plpgsql
SECURITY DEFINER;
588
589  DROP TRIGGER IF EXISTS
trg_protect_user_roles ON
public.usuarios;
590  CREATE TRIGGER
trg_protect_user_roles
591    BEFORE UPDATE ON
public.usuarios
592    FOR EACH ROW EXECUTE
FUNCTION
public.protect_user_roles();
593
594  DROP TRIGGER IF EXISTS
trg_protect_profiles_role ON
public.profiles;
595  CREATE TRIGGER
trg_protect_profiles_role
596    BEFORE UPDATE ON
public.profiles
597    FOR EACH ROW EXECUTE
FUNCTION
public.protect_user_roles();
598
599  -- Solicitações de
Credenciamento de Professores
600  CREATE TABLE IF NOT
EXISTS public.solicitacoes_prof
essores (
601    id UUID DEFAULT
gen_random_uuid() PRIMARY KEY,
602    user_id TEXT NOT NULL,
603    nome TEXT NOT NULL,
604    email TEXT NOT NULL,
605    instrumento TEXT NOT
NULL,
606    bio TEXT,
607    status TEXT DEFAULT
'pendente' CHECK (status IN
('pendente', 'aprovado',
'rejeitado')),
608    decided_at TIMESTAMPTZ,
609    decided_by TEXT,
610    created_at TIMESTAMPTZ
DEFAULT NOW()
611  );
612
613  CREATE INDEX IF NOT
EXISTS idx_solicitacoes_status
ON public.solicitacoes_professo
res(status);
614  CREATE INDEX IF NOT
EXISTS idx_solicitacoes_email
ON public.solicitacoes_professo
res(email);
615  -- Impede solicitações
pendentes duplicadas para o
mesmo e-mail
616  CREATE UNIQUE INDEX IF
NOT EXISTS
idx_solicitacao_pendente_unica
ON public.solicitacoes_professo
res(email) WHERE status =
'pendente';
617
618  -- Marcações de Aulas
Individuais (Professor <->
Aluno)
619  CREATE TABLE IF NOT
EXISTS public.marcacoes_aulas (
620    id UUID DEFAULT
gen_random_uuid() PRIMARY KEY,
621    aluno_id TEXT,
622    aluno_nome TEXT NOT
NULL,
623    aluno_email TEXT NOT
NULL,
624    aluno_telefone TEXT,
625    professor_id TEXT,
626    professor_nome TEXT NOT
NULL,
627    professor_email TEXT,
628    data_aula DATE NOT NULL,
629    hora_inicio TEXT NOT
NULL, -- '14:00'
630    duracao_minutos INTEGER
DEFAULT 50,
631    status TEXT DEFAULT
```

```
'agendada' CHECK (status IN
('agendada', 'realizada',
'cancelada')),
    observacoes TEXT,
    link_meet TEXT DEFAULT
'https://meet.google.com/tut-
aulas-musica',
    created_at TIMESTAMPTZ
DEFAULT NOW()
  );

  CREATE INDEX IF NOT
EXISTS idx_marcacoes_data ON pu
blic.marcacoes_aulas(data_aula)
;
  CREATE INDEX IF NOT
EXISTS
idx_marcacoes_aluno_email ON pu
blic.marcacoes_aulas(aluno_emai
l);
  CREATE INDEX IF NOT
EXISTS idx_marcacoes_prof_nome
ON public.marcacoes_aulas(profe
ssor_nome);

  -- Eventos de Auditoria e
Relatórios de Acesso
  CREATE TABLE IF NOT
EXISTS public.eventos_acesso (
    id UUID DEFAULT
gen_random_uuid() PRIMARY KEY,
    user_id TEXT,
    perfil TEXT NOT NULL,
    tipo_evento TEXT NOT
NULL,
    detalhes JSONB DEFAULT
'{}'::jsonb,
    created_at TIMESTAMPTZ
DEFAULT NOW()
  );

  CREATE INDEX IF NOT
EXISTS idx_eventos_created ON p
ublic.eventos_acesso(created_at
);

  -- Conteúdos Editáveis do
Site (Gerenciados pelo
Desenvolvedor)
  CREATE TABLE IF NOT
EXISTS public.conteudos_site (
    id TEXT PRIMARY KEY,
    titulo TEXT NOT NULL,
    subtitulo TEXT,
    conteudo TEXT,
    secao TEXT NOT NULL, --
'hero', 'anuncio', 'aviso',
'repertorio'
    publicado BOOLEAN
DEFAULT true,
    updated_at TIMESTAMPTZ
DEFAULT NOW()
  );

  -- ====================
==========================
====================

  -- 13. POLÍTICAS DE RLS
GRANULARES PARA OS 3 AMBIENTES

  -- ====================
==========================
====================

  ALTER TABLE public.adminis
tradores_autorizados ENABLE
ROW LEVEL SECURITY;
  ALTER TABLE public.solicit
acoes_professores ENABLE ROW
LEVEL SECURITY;
  ALTER TABLE
public.marcacoes_aulas ENABLE
ROW LEVEL SECURITY;
  ALTER TABLE
public.eventos_acesso ENABLE
ROW LEVEL SECURITY;
  ALTER TABLE
public.conteudos_site ENABLE
ROW LEVEL SECURITY;

  -- 13.1 Administradores
Autorizados
  DROP POLICY IF EXISTS
"Leitura de administradores
por admin" ON public.administra
dores_autorizados;
  CREATE POLICY "Leitura de
```

```
administradores por admin"
    ON public.administradore
s_autorizados FOR SELECT
    USING
(public.is_admin());

-- 13.2 Solicitações de
Professores
-- Alunos só podem criar
solicitação para si próprios e
sempre com status inicial
'pendente'
DROP POLICY IF EXISTS
"Public ou Alunos podem criar
solicitacao de professor" ON pu
blic.solicitacoes_professores;
CREATE POLICY "Public ou
Alunos podem criar solicitacao
de professor"
    ON public.solicitacoes_p
rofessores FOR INSERT
    WITH CHECK (
      status = 'pendente'
      AND (auth.uid() IS
NULL OR auth.uid()::text =
user_id)
    );

-- Consulta de
solicitações: Admin vê todas;
o solicitante vê apenas a sua
própria
DROP POLICY IF EXISTS
"Desenvolvedor gerencia
solicitacoes" ON public.solicit
acoes_professores;
CREATE POLICY
"Desenvolvedor gerencia
solicitacoes"
    ON public.solicitacoes_p
rofessores FOR SELECT
    USING (
      public.is_admin()
      OR auth.uid()::text =
user_id
      OR LOWER(email) =
LOWER(auth.jwt() ->> 'email')
    );

-- Atualização e Decisão
(Aprovar / Rejeitar):
EXCLUSIVA DO DESENVOLVEDOR
-- Usuários comuns NUNCA
podem alterar o status de sua
própria solicitação!
CREATE POLICY "Apenas
desenvolvedor decide
solicitacoes"
    ON public.solicitacoes_p
rofessores FOR UPDATE
    USING
(public.is_admin())
    WITH CHECK
(public.is_admin());

-- 13.3 Marcações de
Aulas (Privacidade estrita
entre Professores e Alunos)
-- Visualização:
Desenvolvedor vê todas;
Professor vê apenas as suas;
Aluno vê apenas as suas
DROP POLICY IF EXISTS
"Acesso as marcacoes de aula"
ON public.marcacoes_aulas;
CREATE POLICY "Acesso as
marcacoes de aula"
    ON
public.marcacoes_aulas FOR
SELECT
    USING (
      public.is_admin()
      OR (auth.uid() IS NOT
NULL AND (professor_id =
auth.uid()::text OR
LOWER(professor_email) =
LOWER(auth.jwt() ->> 'email')))
      OR (auth.uid() IS NOT
NULL AND (aluno_id =
auth.uid()::text OR
LOWER(aluno_email) =
LOWER(auth.jwt() ->> 'email')))
    );

-- Inserção: Apenas
Desenvolvedor ou Professor
```

```
autenticado agendando com seus
alunos
CREATE POLICY
"Professores criam marcacoes
autorizadas"
    ON
public.marcacoes_aulas FOR
INSERT
    WITH CHECK (
      public.is_admin()
      OR (
        auth.uid() IS NOT
NULL
        AND (professor_id =
auth.uid()::text OR
LOWER(professor_email) =
LOWER(auth.jwt() ->> 'email'))
      )
    );

-- Atualização: Apenas
Desenvolvedor ou o Professor
responsável pela aula
CREATE POLICY
"Professores atualizam apenas
suas aulas"
    ON
public.marcacoes_aulas FOR
UPDATE
    USING (
      public.is_admin()
      OR (
        auth.uid() IS NOT
NULL
        AND (professor_id =
auth.uid()::text OR
LOWER(professor_email) =
LOWER(auth.jwt() ->> 'email'))
      )
    );

-- 13.4 Auditoria e
Eventos de Acesso
DROP POLICY IF EXISTS
"Insercao livre de eventos" ON
public.eventos_acesso;
CREATE POLICY "Insercao
livre de eventos"
    ON
public.eventos_acesso FOR
INSERT
    WITH CHECK (true);

DROP POLICY IF EXISTS
"Leitura de eventos por
desenvolvedor" ON
public.eventos_acesso;
CREATE POLICY "Leitura de
eventos por desenvolvedor"
    ON
public.eventos_acesso FOR
SELECT
    USING
(public.is_admin());

-- 13.5 Conteúdos
Editáveis do Site
DROP POLICY IF EXISTS
"Leitura publica de conteudos"
ON public.conteudos_site;
CREATE POLICY "Leitura
publica de conteudos"
    ON
public.conteudos_site FOR
SELECT
    USING (publicado = true
OR public.is_admin());

DROP POLICY IF EXISTS
"Desenvolvedor altera
conteudos" ON
public.conteudos_site;
CREATE POLICY
"Desenvolvedor altera
conteudos"
    ON
public.conteudos_site FOR ALL
    USING
(public.is_admin())
    WITH CHECK
(public.is_admin());

-- ======================
============================
======================
```

```
-- 14. FUNÇÕES RPC SEGURAS (BACKEND STORED PROCEDURES)
-- ============================================================

-- RPC: Aprovação de Professor pelo Desenvolvedor
CREATE OR REPLACE FUNCTION public.aprovar_professor_rpc(p_solicitacao_id UUID)
RETURNS JSONB AS $$
DECLARE
  v_user_id TEXT;
  v_email TEXT;
  v_nome TEXT;
BEGIN
  IF NOT public.is_admin() THEN
    RAISE EXCEPTION 'Acesso negado: apenas o desenvolvedor autorizado pode aprovar professores.';
  END IF;

  UPDATE public.solicitacoes_professores
  SET status = 'aprovado',
      decided_at = NOW(),
      decided_by = COALESCE(auth.jwt() ->> 'email', 'Desenvolvedor Autorizado')
  WHERE id = p_solicitacao_id
  RETURNING user_id, email, nome INTO v_user_id, v_email, v_nome;

  IF NOT FOUND THEN
    RAISE EXCEPTION 'Solicitação de professor não encontrada.';
  END IF;

  -- Promove o usuário na tabela usuarios
  IF v_user_id IS NOT NULL AND v_user_id <> 'anonimo' THEN
    UPDATE public.usuarios
    SET role = 'teacher',
        approval_status = 'aprovado',
        updated_at = NOW()
    WHERE user_id::text = v_user_id;

    UPDATE public.profiles
    SET role = 'teacher',
        approval_status = 'aprovado',
        updated_at = NOW()
    WHERE id::text = v_user_id;
  END IF;

  -- Registra na auditoria
  INSERT INTO public.eventos_acesso (user_id, perfil, tipo_evento, detalhes)
  VALUES (
    auth.uid()::text,
    'desenvolvedor',
    'professor_aprovado',
    jsonb_build_object('solicitacao_id', p_solicitacao_id, 'email_professor', v_email, 'nome', v_nome)
  );

  RETURN jsonb_build_object('success', true, 'status', 'aprovado', 'professor_email', v_email);
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- RPC: Rejeição de Professor pelo Desenvolvedor
CREATE OR REPLACE
```

```
FUNCTION public.rejeitar_profes
sor_rpc(p_solicitacao_id UUID)
819  RETURNS JSONB AS $$
820  DECLARE
821    v_user_id TEXT;
822    v_email TEXT;
823  BEGIN
824    IF NOT
public.is_admin() THEN
825      RAISE EXCEPTION
'Acesso negado: apenas o
desenvolvedor autorizado pode
rejeitar professores.';
826    END IF;
827
828    UPDATE
public.solicitacoes_professores
829    SET status =
'rejeitado',
830        decided_at = NOW(),
831        decided_by =
COALESCE(auth.jwt() ->>
'email', 'Desenvolvedor
Autorizado')
832    WHERE id =
p_solicitacao_id
833    RETURNING user_id,
email INTO v_user_id, v_email;
834
835    IF NOT FOUND THEN
836      RAISE EXCEPTION
'Solicitação de professor não
encontrada.';
837    END IF;
838
839    -- Mantém o usuário
como aluno regular
840    IF v_user_id IS NOT
NULL AND v_user_id <>
'anonimo' THEN
841      UPDATE public.usuarios
842      SET role = 'student',
843          approval_status =
'rejeitado',
844          updated_at = NOW()
845      WHERE user_id::text =
v_user_id;
846
847      UPDATE public.profiles
848      SET role = 'student',
849          approval_status =
'rejeitado',
850          updated_at = NOW()
851      WHERE id::text =
v_user_id;
852    END IF;
853
854    INSERT INTO
public.eventos_acesso
(user_id, perfil, tipo_evento,
detalhes)
855    VALUES (
856      auth.uid()::text,
857      'desenvolvedor',
858      'professor_rejeitado',
859      jsonb_build_object('so
licitacao_id',
p_solicitacao_id,
'email_professor', v_email)
860    );
861
862    RETURN
jsonb_build_object('success',
true, 'status', 'rejeitado',
'professor_email', v_email);
863  END;
864  $$ LANGUAGE plpgsql
SECURITY DEFINER;
865
866  -- Seeds de Demonstração
867  INSERT INTO public.solicit
acoes_professores (nome,
email, instrumento, bio,
status)
868  VALUES
869    ('Profª Clara
Vasconcelos', 'professora.clara
@tutti.mus.br', 'Saxofone
Tenor & Flauta', 'Especialista
em Bossa Nova e MPB formada
pela EMESP.', 'aprovado'),
870    ('Prof. Marcos
Drumond',
'marcos.drumond@musica.br',
'Trompete & Metais', 'Spalla
de trompete da orquestra
```

```
sinfônica.', 'pendente'),
    ('Prof. Thiago
Alcantara',
'thiago.alcantara@gmail.com',
'Violão & Canto', 'Professor
particular de harmonia e
violão 7 cordas.', 'pendente')
  ON CONFLICT DO NOTHING;

INSERT INTO
public.marcacoes_aulas
(aluno_nome, aluno_email,
aluno_telefone,
professor_nome, data_aula,
hora_inicio, duracao_minutos,
status, observacoes)
  VALUES
    ('Rafael Drumond',
'rafael.aluno@tutti.mus.br',
'(11) 98844-2211', 'Profª
Clara Vasconcelos',
CURRENT_DATE + INTERVAL '1
day', '15:00', 50, 'agendada',
'Estudo do solo de Autumn
Leaves e afinação nas notas
graves do Sax Alto.'),
    ('Beatriz Helena',
'beatriz.helena@gmail.com',
'(21) 97722-3344', 'Profª
Clara Vasconcelos',
CURRENT_DATE + INTERVAL '2
days', '16:00', 50,
'agendada', 'Flauta
transversal: respiração
diafragmática e apoio sonoro
em Carinhoso.'),
    ('Lucas Gabriel',
'lucas.gabriel@gmail.com',
'(31) 99112-4455', 'Maestro
Lucas', CURRENT_DATE +
INTERVAL '3 days', '10:00',
50, 'agendada', 'Harmonia
funcional: cadência ii-V-I e
substituições tritônicas.')
  ON CONFLICT DO NOTHING;

INSERT INTO
public.conteudos_site (id,
titulo, subtitulo, conteudo,
secao)
  VALUES
    ('banner_principal',
'Domine o seu Instrumento com
a Tutti', 'Aulas ao vivo,
estúdio interativo e
acompanhamento pedagógico com
o Maestro Lucas.', 'O método
Tutti integra teoria clássica
com a musicalidade
brasileira.', 'hero'),
    ('aviso_aulas', 'Aulas
Particulares com o Google
Meet', 'Cada aluno recebe link
exclusivo de videoconferência
para sua aula individual.',
'Garantia de atendimento
individualizado.', 'aviso')
  ON CONFLICT (id) DO
NOTHING;

-- ======================
==============================
======================

-- FIM DO SCRIPT DE
ARQUITETURA TUTTI &j

-- ======================
==============================
======================
```